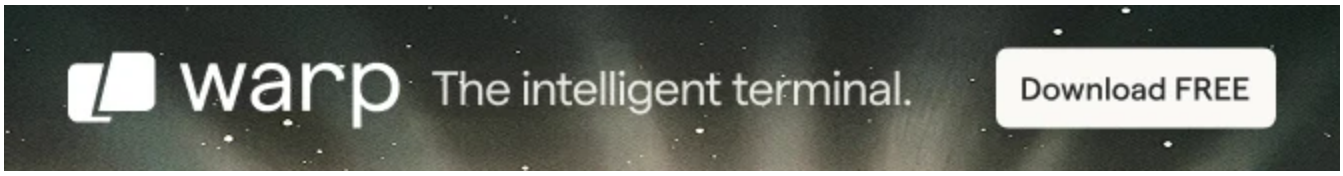
Popular Open Source Licenses and How to Choose Them

This detailed guide gives you an effective Open Source license comparison. It should help you choose the right Open Source license for your project.

Sylvain Leroux
20 Jan 2025 · 11 min read · 5 comentarios

ON THIS PAGE

Do I need a License? And what is a License after all?
What is the purpose of the License?
And what if I don't use any License at all?
Can I use a custom License?
May I use several Licenses?
Can I change the License "later"?
Which Open Source License should I use?
GNU General Public License (GPL)
GNU Lesser General Public License (LGPL)
Eclipse Public License (EPL 1.0)
Mozilla Public License (MPL)
Apache License 2.0 (ASL 2.0)
MIT License
BSD Licenses
The last word on Open Source licenses



So, you're working on that cool new project for a while — and you're ready now to make the critical move from *closed source* to *open source*.

Subscribe

It doesn't seem much more work than cleaning the sources and the commit history before pushing your repository onto GitHub or Bitbucket… … until the question of the License pops out. There are so many choices available. Which one to choose? And do you *really* need a License after all?

The short answer to that latter question is easy: Yes, you *really* need a License. As about what License you need, I can even make a shorter answer: *it depends*.

But if you're serious about your project, you probably want a little bit more details. So read ahead — and remember: you're entering holy war territory now!

Do I need a License? And what is a License after all?

A License is an *official* permission granted by the owner of some Work (the "Licensor") to other people (the "Licensee") and governing how the Licensee is allowed to use the Licensor's Work.

This takes the form of a contract, both parties has to agree with. Nowadays, the acceptation is rather implicit: just by *using* some Work, you are reputed to agree with its usage License.

Just to make thinks clear, when releasing your *own* work, the Licensor is *you*. And the Licensee, *anyone* using your code. Broadly speaking, this includes two main categories: *developers* and *end users*.

And to fix few more vocabulary terms, by *modifying* your Work, a Licensee is creating what is called a Derivative Work. Not all licenses agree though if the *use* of your Work in a larger work will qualify that latter as a Derivative Work or not. As you will see below, some licenses specifically address those issues.

What is the purpose of the License?

Basically, the License is a way for the Licensor and the Licensee to agree on the *rights and obligations* of *both* of them. Those rights and obligations associated with a License can be anything — *up to the extent of what's allowed by the Law*. For example, a Licensor might require the Licensee to quote her name when using her work. Or can authorize to copy her work, but not to

modify it in any way. Or even require Derivative Work to be released under the same terms as the original Work.

On the other hand, the License is a way to protect the Licensee too. By clearly stating how he can use your Work, he is not at risk of seeing you unexpectedly asking for royalties or another form of compensation for having used your work. Something that is critical for your Work adoption.

So, the License will protect your work. Will protect the Licensor. But will protect you too. I mean you, *personally*. For example by limiting the Licensor's responsibility for potential damages caused by her work.

And what if I don't use any License at all?

In absence of a License explicitly associated with a Work, the "default" copyright for the author's jurisdiction applies. In other words, *never* consider the "absence of license" as an implicit grant for us to do whatever we want with your work. This is the exact opposite: without any specific license, you, the author, didn't waive ANY of your rights as granted by the law.

But always remember a License governs rights *and* obligations. Did you ever wonder why so many License text contains a disclaimer written in ALL UPPERCASE LETTERS about the guarantees provided with a product — or more often the absence of guaranty? This is to *protect* the owner of the work against implicit guarantees or user assumptions. The last thing you want is to be sued as a consequence of releasing your work open-source!

Can I use a custom License?

Yes, you can. But you probably shouldn't.

Being a contract, a License cannot (in most jurisdictions? all of them?) take precedence over the territorial laws. Hence the difficulty enforcing the licensing rights in a globalized world. It would be probably easier (I mean, less difficult) to defend a "standard" License before a judge. In fact, such cases have already been defended in several jurisdictions and may be cited as precedent. Obviously, something that can't be done with a Custom License.

In additions, Custom Licenses (sometimes nicknamed Vanity Licenses) may create incompatibilities with other licenses, resulting in a poor compatibility of your Work legally speaking.

May I use several Licenses?

Yes. Multi-licensing — notably Dual-licensing — is not that uncommon. This is especially true when you want to build a business around your free Work. In that case, your project will probably be released both under some FOSS license and a commercial license.

Another use of multi-licensing is to increase compatibility, by allowing your Work to be combined with works published under different terms or to satisfy different user needs or requirements. This is a reason why some project are released under several FOSS licenses.

But be warned: not all licenses are compatible together! Once again, I would discourage you from reinventing the wheel by staying with well-known compatible licenses if you want to go that way.

Can I change the License "later"?

Yes. The copyright holder is responsible for the licensing terms. It is rather easy to change the License as long as you are the only contributor. But to take an extreme example, if Linus Torvald would want to release the Linux Kernel under a different license, he would probably need first the agreement of the thousands of contributors to that project. Something impossible in practice.

For a more reasonably sized project, it can be done. And in fact, it was as you will see in some examples below.

Which Open Source License should I use?

Ok, now, you're convinced you should use a Standard License. But Which one to choose? The final choice is up to you. And there are very well made comparators available on the web to help you in your choice. Just to quote my favorites:

- http://oss.ly/licdif
- https://choosealicense.com/ / https://choosealicense.com/appendix/

- https://opensource.org/licenses
- https://tldrlegal.com/

But as always with legal affairs, the definitive answer will be to read — and understand — the authoritative text of the License. That may require the help of a professional lawyer. Something I am not.

But what I can do is providing you some introduction to the most common Licenses in order to guide your first steps.

GNU General Public License (GPL)

The GPL is one of the most popular Open Source License. It comes in several versions — but for a new project, you should consider the most recent, which is the GPL 3 at the time of this writing.

Supporting a strong copyleft, the GPL is probably the most protective free software license. Something it can be praised or criticized for depending your point of view. The core concept behind the GPL being *any* Derivative Work must be released under the GPL too.

- **Strong copyleft**
- The Work is suitable for commercial use.
- Licensees can modify the work.
- Licensees must release the source alongside with Derivative Work.
- Derivative Work must be released under the same terms.

Popular projects

The GPL is the natural license for the projects of the Free Software Foundation. Including the GNU tools at the heart of any Linux system. Large projects — *a fortiori* commercial ones — tend to use the GPL in conjunction with one or several other Licenses.

- Inkscape (Vector drawing): GPLv2
- Drupal (Web Content Management System): GPLv2

- MariaDB (Databases): GPL v2
- MySQL (Databases): GPL and Commercial License
- Qt (cross-platform application framework): LGPL, GPL, and Commercial — depending the modules and service-agreement level

GNU Lesser General Public License (LGPL)

The GPL is very restrictive in the sense it forces any Derivative Work to be released open-source under the same terms. This is especially a concern for libraries — which are building blocks for larger software: by releasing a library under the GPL, you will force any application *using* that library to be released as GPL too. Something the LGPL addresses.

For libraries, the FSF distinguishes three cases:

- Your library implements a standard that competes with a non-free standard. In that case, wide adoption of your library will help the Free Software cause. The FSF suggests the quite permissive Apache License for that case (described later in that article).
- Your library implements a standard already implemented by other libraries. In that case, there is no benefit for the Free Software cause to abandon copyleft entirely. So the FSF recommends the LGPL.
- Finally, if your library does *not* compete with other libraries nor other standards, the FSF recommends the GPL.

The FSF arguments are mostly ethical and philosophical. In practice, developers may have other concerns. Especially if they plan to develop a business based on the licensed work. Once again, dual-licensing may be an option to consider.

- **Weak copyleft (bound to dynamically linked library)**
- The Work is suitable for commercial use.
- Licensees can modify the work.
- Licensees must release the source alongside with Derivative Work.
- if you *modify* the Work, you *must* release the Modified Work under the same terms.
- if you *use* the Work, you _don't need_to release the Derivative Work under the same terms.

Popular projects

- OpenOffice.org 3 (office suite): LGPLv3 — but Apache OpenOffice 4 switched to Apache License 2.0.
- GTK+, the GIMP Toolkit (GUI toolkit): LGPLv2.1
- CUPS (cross-platform printing system): GPL or LGPLv2 with an exception for Apple operating systems — depending the components.
- WineHQ (Windows compatibility layer): LGPLv2.1
- GNU Aspell (Spell checker): LGPLv2.1

Eclipse Public License (EPL 1.0)

With a weaker copyleft than the LGPL, the Eclipse License is more Business-Friendly as it allows sub-licensing and building of software made of EPL and non-EPL (even proprietary) licensed code, provided the non-EPL code is a *"separate module[s] of software"*.

In addition, the EPL adds extra protection for the EPL code contributors in the case of lawsuits/damages caused by a commercial offering including that Work.

- **Weak copyleft (bound to software "module")**
- The Work is suitable for commercial use.
- The Licensees can modify the work.
- If you *modify* the Work, you *must* release the Modified Work under the same terms.
- If you *use* the Work, you _don't need_to release the Derivative Work under the same terms.

- Commercial distributors of the software must defend or compensate original EPL contributors from lawsuits/damages caused by the commercial offering.

Popular projects

Obviously, the EPL is the natural license for the projects of the Eclipse Foundation. Including the popular Eclipse IDE. But it gained some popularity beyond that — especially in the Java world:

- Clojure (Programming language)
- Graphviz (Graph visualization package)
- Jetty (Application server): dual license EPL1.0/Apache License 2.0 since Jetty 7
- JUnit (Java unit testing framework)

Mozilla Public License (MPL)

The Mozilla Public License is license used for software developed by the Mozilla foundation. But it's certainly not limited to that area. The MPL aims at being a compromise step between strict licenses (like the GPL) and permissive licenses (like the MIT License).

In the MPL the "licensing unit" is the source file. Licensors are not allowed to restrict the user rights and access on any file covered by the MPL. But the same project can also contain proprietary non-MPL licensed files. The resulting project can be released under any license, provided that access to the MPL licensed files is granted.

- **Weak copyleft (bound to individual files)**
- The Work is suitable for commercial use.
- Licensees can modify the work.
- Licensees must provide proper attribution for the Work.
- Licensees may redistribute Derivative Work under different terms
- Licensees cannot relicense MPL-licensed source
- Licensees must distribute MPL-licensed source code alongside with their Derivative Work.

Popular projects

- Mozilla Firefox(web browser), Mozilla Thunderbird(email client): MPL
- LibreOffice (office suite): MPL2.0
- H2 Database Engine (database): MPL2.0 and Eclipse License 1.0
- Cairo (2D graphic engine): MPL 1.1 or LGPLv2.1

Apache License 2.0 (ASL 2.0)

With the ASL, we are entering the realm of *permissive* free licenses. But even the FSF suggest Apache License in some cases. The Apache License is permissive as it does not require *any* Derivative Work to be distributed under the same terms. In other words, this is a non-copyleft license.

The ASL is the only license used for projects of the Apache Software Foundation. Being considered as business-friendly, it has gained widespread adoption outside of that organization. It is not uncommon to see enterprise-grade projects to be released under the ASL.

- **Non-copyleft**
- The Work is suitable for commercial use.
- Licensees can modify the work.
- Licensees must provide proper attribution for the Work.
- Licensees may redistribute Derivative Work under different terms.
- Licensees do not have to distribute the source code alongside with their Derivative Work.

Popular projects

- Android (operating system): ASL 2.0 with some exceptions (notably regarding the Linux kernel)
- Apache httpd (Web server): ASL 2.0
- Apache Spark (Cluster computing framework): ASL 2.0
- Spring Framework (Framework for Java-based enterprise applications): ASL 2.0

MIT License

This one is a very popular license. Even probably the most popular one. By putting very few limitations on reuse, the MIT License can easily be associated with other licenses, from the GPL to proprietary licenses.

- **Non-copyleft**
- The Work is suitable for commercial use.
- Licensees can modify the work.
- Licensees must provide proper attribution for the Work.
- Licensees may redistribute Derivative Work under different terms
- Licensees do not have to distribute the source code alongside with their Derivative Work.

Popular projects

- node.js (JavaScript runtime environment): MIT License
- jQuery (client-side JavaScript library): MIT License (until 2012, dual-license MIT/GPL)
- Atom (text editor): MIT License
- AngularJS (JavaScript application framework): MIT License
- SQLAlchemy (SQL toolkit and Object Relational Mapper for Python): MIT License

BSD Licenses

BSD License comes in three flavors. The original 4-clause License, the “revised” 3-clause License and the “simplified” 2-clause License. All in spirit are very close to the MIT License. And indeed, there is very little practical differences between the 2-clause BSD License & the MIT License.

3- and 4-clause BSD Licenses add more requirements concerning name reuse and advertising. This is something to consider if you want to protect your product or brand name.

- **Non-copyleft**
- The Work is suitable for commercial use.
- Licensees can modify the work.
- Licensees must provide proper attribution for the Work.
- Licensees may redistribute Derivative Work under different terms.
- Licensees do not have to distribute the source code alongside with their Derivative Work.
- Licensees cannot use the original Author name or trademark to endorse Derivative Work (3- and 4- clause BSD)
- Licensees must acknowledge the original Author in all advertising materials mentioning features or use of the Work (4-clause BSD)

Popular projects

- Django (web ramework): 3-clause BSD
- Redis (data store): 3-clause BSD

- Ruby (programming language): 2-clause BSD and custom license
- Nginx (Web server): 2-clause BSD
- NetBSD (Operating system): 2-clause BSD — 4-clause BSD until 2008

The last word on Open Source licenses

If you come that far, congratulations! You understand it now, *licensing* is really a huge *and* complex topic. But it worth taking the time to choose the right license for your project — and to make that choice early. It could save you lots of problems later, so you can use your time and energy working on your project rather than dealing with copyright or legal compatibility issues.

Even if I've done my best to make that topic accessible, it is not always easy to summarize the subtleties of the various licenses. And beyond the few major licenses presented here, there are dozens of others more or less commonly used.

So, don't hesitate to use the comment section below to tell us what's *YOUR* preferred license and why. Or to mention some important characteristics I might have forgotten!

Guides 📒 Open Source

ABOUT THE AUTHOR

Sylvain Leroux

Engineer by Passion, Teacher by Vocation. My goals : to share my enthusiasm for what I teach and prepare my students to develop their skills by themselves. You can

find me on <a href="https://www.yes

Featured

After 6 Years, One of Wayland's Most Annoying Problems is Finally Getting Fixed

After a Long Time, Ubuntu Shows Some Respect to Deb Packages

Why Ubuntu 26.10 Might Drop ZFS, RAID & Encryption Support

Inside the Systemd Age Verification Debate: Developer Responds to Criticism

Someone Forked Systemd to Strip Out Its Age Verification Support

Latest

Ubuntu 26.04 LTS Beta Shows You There's Potential in the Stable Release

27 Mar 2026

FOSS Weekly #26.13: Age Verification Added in systemd, Systemd forked, Btrfs Subvolumes, New Backup Tool, Yazi Manager and More

26 Mar 2026

Troubleshooting "AppImages require FUSE to run" on Linux

26 Mar 2026

Become a Better Linux User

With the FOSS Weekly Newsletter, you learn useful Linux tips, discover applications, explore new distros and stay updated with the latest from Linux world

Your email address

Subscribe

Enter your comment here. No hate, abuse, or NSFW content. Be respectful even in disagreement. All comments are manually reviewed and may take a few hours to appear.

Enviar respuesta

GIF

Sign in to your member account

Commenting is better for members. You can vote, get reply notifications, and earn karma and badges. Sign in to enjoy it all, or comment as a guest (without the perks).

Your Email Address (Won't be published here)

Nombre de usuario

You may receive notification if you receive a reply to your message

Enviar

5 comentarios

El más reciente primero

asonte 2 hace años

To call the GNU GPL ‘one of the most popular Open Source License’ is nonsense and shows that you are utterly confused. The GNU GPL is about protecting the users FREEDOM. It is a LIBRE software license. I’m not going to rant here, just listen to the talks of Richard Stallman and do your research.

Respuesta

John 5 hace años

There is also: https://joinup.ec.europa.eu/collection/eupl/solution/joinup-licensing-assistant/jla-find-and-compare-software-licenses Which is pretty awesome and functional

Respuesta

Ravi 6 hace años

If the opensource component has dual license association like GPL & MIT, how this can be used to avoid license implications ? Shall we use it under MIT by having the copyright info related to MIT only?

Respuesta

Ak 8 hace años

We know every day many software released, to maintain the copyrights and patents of the software this license used under some condition. In today's world there are many open source license, some of the famous or important licenses are : APACHE LICENSE BSD LICENSE GPL LICENSE LGPL LICENSE

Respuesta

Giovanni Baruzzi 8 hace años

thank you for your valuable contribution! I am still lost looking for a OpenSource Licente that fits our needs but this is a very good first step.

Respuesta

FastComments.com

READ NEXT

GitHub's 2025 Report Reveals Some Surprising Developer Trends

FFmpeg Receives $100K in Funding from India's FLOSS/fund Initiative

Good News! Austrian Ministry Kicks Out Microsoft in Favor of Nextcloud

Community Strikes Back: 12 Open Source Projects Born from Resistance

Top 5 FOSS Philosophies Creatives Should Embrace

Become a Better Linux User

With the FOSS Weekly Newsletter, you learn useful Linux tips, discover applications, explore new distros and stay updated with the latest from Linux world

Your email address

SUBSCRIBE

Making You A Better Linux User

Your email address

Subscribe

Navigation

©2026 It's FOSS. Hosted on Digital Ocean (get $200 in FREE credits) & Published with Ghost & Rinne.

System